

Chapter 5: First-Order Methods

Algorithms for Optimization, 2nd ed. (2026)

Kochenderfer & Wheeler

Lecture Notes

June 12, 2026

Table of Contents

- 1 Introduction to First-Order Methods
- 2 5.1 Gradient Descent
- 3 5.2 Conjugate Gradient Method
- 4 5.3 Momentum (Heavy Ball)
- 5 5.4 Nesterov Momentum
- 6 5.5 AdaGrad
- 7 5.6 RMSProp
- 8 5.7 Adadelta
- 9 5.8 Adam
- 10 5.9 Hypergradient Descent
- 11 Convergence Comparison
- 12 Summary & Exercises

What Are First-Order Methods?

Definition

First-order methods use gradient information $\nabla f(\mathbf{x})$ to select a descent direction.

Why gradient information?

- Gradient points in the direction of *steepest ascent*
- Opposite direction is the direction of *steepest descent*
- Computing ∇f costs $O(n)$ for most functions (vs. $O(n^2)$ for Hessian)
- Scalable to high-dimensional problems (e.g. deep learning)

Chapter Roadmap

- 1 Gradient Descent (steepest descent)
- 2 Conjugate Gradient Method
- 3 Momentum (Heavy Ball)
- 4 Nesterov Momentum
- 5 AdaGrad
- 6 RMSProp
- 7 Adadelta
- 8 Adam
- 9 Hypergradient Descent

Requirement

f must be differentiable (or sub-differentiable for

5.1 Gradient Descent — Motivation

Define the gradient at the k -th iterate $\mathbf{x}^{(k)}$:

Gradient Notation (Eq. 5.1)

$$\mathbf{g}^{(k)} = \nabla f(\mathbf{x}^{(k)})$$

- $\mathbf{g}^{(k)} \in \mathbb{R}^n$: partial derivatives at $\mathbf{x}^{(k)}$

First-order Taylor approximation about $\mathbf{x}^{(k)}$:

Linear Approximation (Eq. 5.2)

$$f(\mathbf{x}^{(k)} + \alpha \mathbf{d}) \approx f(\mathbf{x}^{(k)}) + \alpha \mathbf{d}^\top \mathbf{g}^{(k)}$$

- α : step factor (learning rate)
- $\mathbf{d}$: unit descent direction ($\|\mathbf{d}\| = 1$)

Minimizing over $\mathbf{d}$ subject to $\|\mathbf{d}\| = 1$ gives the *steepest*

Update rule:

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - \alpha \mathbf{g}^{(k)}$$

(many implementations skip normalisation;
then α is not the step *length*)

Key insight: moving *opposite* the gradient
decreases f locally.

Note (L_2 norm)

Using the L_2 norm gives steepest descent.
Using the L_1 norm gives *coordinate descent*.

5.1 Gradient Descent — Algorithm and Python Code

Algorithm: Gradient Descent

Input: f , ∇f , initial $\mathbf{x}^{(1)}$, step α , iterations T

For $k = 1, 2, \dots, T$:

- 1 Compute $\mathbf{g}^{(k)} = \nabla f(\mathbf{x}^{(k)})$
- 2 Normalise: $\mathbf{d}^{(k)} = -\mathbf{g}^{(k)} / \|\mathbf{g}^{(k)}\|$
- 3 Update: $\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \alpha \mathbf{d}^{(k)}$

Return $\mathbf{x}^{(T+1)}$

Cost per iteration: 1 gradient evaluation, $O(n)$

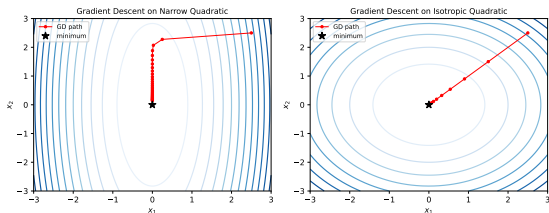
Convergence (for L -smooth, μ -strongly convex f):

$$\|\mathbf{x}^{(k)} - \mathbf{x}^*\|^2 \leq \left(1 - \frac{\mu}{L}\right)^k \|\mathbf{x}^{(1)} - \mathbf{x}^*\|^2$$

Linear convergence rate $\kappa = \mu/L$ (condition number).

```
1 import numpy as np
2
3 def gradient_descent(f, grad_f, x, alpha,
4                     n_iter=100, normalize=True)
5     :
6     """
7     f           : objective function
8     grad_f      : gradient of f
9     x           : initial point (1D numpy array)
10    alpha        : step factor
11    normalize    : use unit step direction
12    """
13    path = [x.copy()]
14    for _ in range(n_iter):
15        g = grad_f(x)
16        if normalize:
17            d = -g / (np.linalg.norm(g) + 1e-15)
18        else:
19            d = -g # unnormalized
20        x = x + alpha * d
21        path.append(x.copy())
22    return x, np.array(path)
23
24 # Example: f(x) = x1^2 * x2
25 f = lambda x: x[0]**2 * x[1]
26 gradf = lambda x: np.array([2*x[0]*x[1], x[0]**2])
```

5.1 Gradient Descent — Paths and Numerical Example



Left: zigzag on ill-conditioned quadratic. Right: smooth convergence on isotropic quadratic.

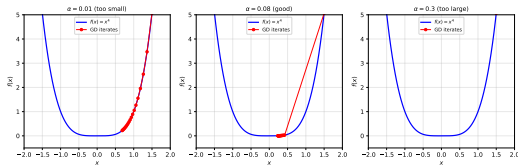
Numerical Example: $f(\mathbf{x}) = x_1 x_2^2$
At $\mathbf{x}^{(k)} = [1, 2]$, $\alpha = 0.1$ (normalised):

$$\nabla f = [x_2^2, 2x_1 x_2] = [4, 4]$$

$$\mathbf{d}^{(k)} = -\frac{[4, 4]}{\sqrt{32}} = \left[-\frac{1}{\sqrt{2}}, -\frac{1}{\sqrt{2}}\right]$$

$$\mathbf{x}^{(k+1)} = [1, 2] + 0.1 \left[-\frac{1}{\sqrt{2}}, -\frac{1}{\sqrt{2}}\right] \approx [0.929, 1.929]$$

Weakness: may zig-zag in narrow valleys. The step size α must be tuned carefully.



small $\Rightarrow$ slow; too large $\Rightarrow$ diverges.

5.2 Conjugate Gradient — Motivation

Problem with gradient descent: on narrow valleys, successive gradients are *nearly orthogonal* $\Rightarrow$ slow zigzagging.

Conjugate gradient (CG) borrows from exact solvers for quadratic objectives:

Quadratic Objective (Eq. 5.8)

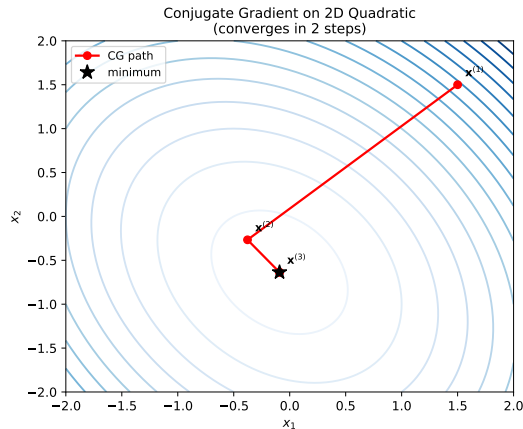
$$\min_{\mathbf{x}} \frac{1}{2} \mathbf{x}^\top \mathbf{A} \mathbf{x} + \mathbf{b}^\top \mathbf{x} + c$$

$\mathbf{A}$ symmetric positive definite $\Rightarrow$ unique minimum.

Key Property: Conjugacy (Eq. 5.9)

Directions $\mathbf{d}^{(1)}, \dots, \mathbf{d}^{(n)} \neq \mathbf{0}$ are **mutually conjugate** w.r.t. $\mathbf{A}$ if:

$$\mathbf{d}^{(i)\top} \mathbf{A} \mathbf{d}^{(j)} = 0 \quad \text{for all } i \neq j$$



Conjugate gradient converges in $n = 2$ steps on a 2-D quadratic. Note: the search directions are *not* orthogonal in general.

5.2 Conjugate Gradient — Update Rules I

Step 1 — initialise with steepest descent:

$$\mathbf{d}^{(1)} = -\mathbf{g}^{(1)} \quad (5.10)$$

Step 2 — exact line search (for quadratic f , Example 5.1):

$$\alpha = -\frac{\mathbf{d}^\top (\mathbf{A}\mathbf{x} + \mathbf{b})}{\mathbf{d}^\top \mathbf{A}\mathbf{d}} = -\frac{\mathbf{d}^\top \mathbf{g}}{\mathbf{d}^\top \mathbf{A}\mathbf{d}}$$
$$\mathbf{x}^{(2)} = \mathbf{x}^{(1)} + \alpha^{(1)} \mathbf{d}^{(1)} \quad (5.11)$$

Step $k > 1$ — update direction (Eq. 5.12):

$$\mathbf{d}^{(k)} = -\mathbf{g}^{(k)} + \beta^{(k)} \mathbf{d}^{(k-1)}$$

Optimal $\beta^{(k)}$ (when $\mathbf{A}$ is known, Eq. 5.16):

$$\beta^{(k)} = \frac{\mathbf{g}^{(k)\top} \mathbf{A} \mathbf{d}^{(k-1)}}{\mathbf{d}^{(k-1)\top} \mathbf{A} \mathbf{d}^{(k-1)}}$$

5.2 Conjugate Gradient — Update Rules II

For general (non-quadratic) functions: several approximations that do *not* require **A**:

Fletcher–Reeves update (Eq. 5.17)

$$\beta_{\text{FR}}^{(k)} = \frac{\mathbf{g}^{(k)\top} \mathbf{g}^{(k)}}{\mathbf{g}^{(k-1)\top} \mathbf{g}^{(k-1)}}$$

Ratio of squared norms of successive gradients.

Polak–Ribière update (Eq. 5.18)

$$\beta_{\text{PR}}^{(k)} = \frac{\mathbf{g}^{(k)\top} (\mathbf{g}^{(k)} - \mathbf{g}^{(k-1)})}{\mathbf{g}^{(k-1)\top} \mathbf{g}^{(k-1)}}$$

Convergence can be guaranteed if $\beta^{(k)} = \max(\beta_{\text{PR}}^{(k)}, 0)$.

In practice: CG is restarted (i.e. $\mathbf{d}^{(k)} \leftarrow -\mathbf{g}^{(k)}$) every n iterations or when $\mathbf{d}^{(k)\top} \mathbf{g}^{(k)} > 0$.

5.2 Conjugate Gradient — Python Implementation

```
import numpy as np

def conjugate_gradient(grad_f, x, n_iter=None, restart=True):
    """
    Non-linear CG with Fletcher-Reeves beta and optional restart.
    grad_f : function returning gradient at x
    x       : initial point (numpy array)
    """
    n = len(x)
    if n_iter is None:
        n_iter = 10 * n
    g = grad_f(x)
    d = -g.copy()                # steepest descent on first step
    path = [x.copy()]

    for k in range(1, n_iter + 1):
        # Simple backtracking line search (Armijo)
        alpha = 1.0
        f0 = None # f not explicitly needed for direction
        while True:
            x_new = x + alpha * d
            g_new = grad_f(x_new)
            # Armijo check (using gradient dot product as proxy)
            if np.dot(g_new, g_new) < np.dot(g, g):
                break
            alpha *= 0.5
        if alpha < 1e-10:
```

5.3 Momentum — Intuition

Problem: gradient descent takes a long time to traverse flat regions or narrow valleys.

Intuition: a ball rolling downhill *accumulates velocity* — it accelerates in consistent directions and dampens oscillations.

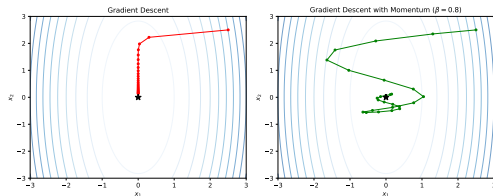
Momentum Update Rule (Eq. 5.20–5.21)

Maintain a **velocity vector** $\mathbf{v}^{(k)}$:

$$\mathbf{v}^{(k+1)} = \beta \mathbf{v}^{(k)} - \alpha \mathbf{g}^{(k)}$$

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \mathbf{v}^{(k+1)}$$

- $\beta \in [0, 1)$: momentum decay (friction)
- α : step factor
- $\mathbf{v}^{(1)} = \mathbf{0}$ (or set to zero on init)



Top: vanilla GD zigzags; bottom: momentum dampens oscillations and accelerates along the valley.

Steady-state velocity (constant gradient):

$$\mathbf{v}^* = \frac{-\alpha}{1-\beta} \mathbf{g}$$

Speed-up factor vs GD: $\frac{1}{1-\beta}$. For $\beta = 0.9$: $10\times$ effective step!

5.3 Momentum — Python Implementation

```
import numpy as np

def momentum_descent(grad_f, x, alpha=0.01, beta=0.9,
                    n_iter=500):
    """
    grad_f : gradient function
    x       : initial point
    alpha   : step factor (learning rate)
    beta    : momentum decay in [0, 1)
    """
    v = np.zeros_like(x)    # velocity initialised to zero
    path = [x.copy()]

    for _ in range(n_iter):
        g = grad_f(x)
        v = beta * v - alpha * g    # update velocity
        x = x + v                  # update position
        path.append(x.copy())

    return x, np.array(path)

# Example: narrow quadratic
A = np.diag([8.0, 1.0])    # ill-conditioned (ratio 8:1)
grad_f = lambda x: A @ x

x0 = np.array([2.5, 2.5])
```

5.4 Nesterov Momentum

Key insight (Nesterov, 1983): standard momentum evaluates the gradient at the *current* position $\mathbf{x}^{(k)}$, but then steps to $\mathbf{x}^{(k)} + \mathbf{v}$.

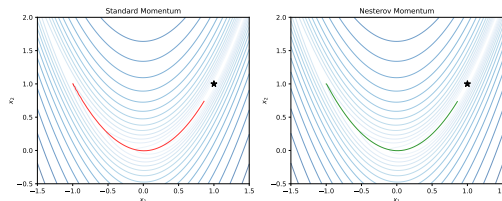
Why not evaluate the gradient at the next position?

Nesterov Update (Algorithm 5.4)

$$\mathbf{v}^{(k+1)} = \beta \mathbf{v}^{(k)} - \alpha \nabla f(\mathbf{x}^{(k)} + \beta \mathbf{v}^{(k)})$$

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \mathbf{v}^{(k+1)}$$

- Gradient evaluated at **lookahead** point $\mathbf{x}^{(k)} + \beta \mathbf{v}^{(k)}$
- Same parameters as momentum: α, β
- Extra gradient call per step (same asymptotic cost)



Nesterov (right) typically overshoots less and converges faster on the Rosenbrock function.

Intuition: Nesterov “looks ahead” to where it will be, so it can slow down *before* overshooting.

5.4 Nesterov Momentum — Python Code

```
import numpy as np

def nesterov_momentum(grad_f, x, alpha=0.001, beta=0.9,
                      n_iter=500):
    """
    Nesterov accelerated gradient descent.
    grad_f : gradient function
    x       : initial point
    alpha   : learning rate
    beta    : momentum decay in [0, 1)
    """
    v = np.zeros_like(x)
    path = [x.copy()]

    for _ in range(n_iter):
        # Evaluate gradient at LOOKAHEAD position
        g = grad_f(x + beta * v)
        v = beta * v - alpha * g    # update velocity with lookahead grad
        x = x + v                  # update position
        path.append(x.copy())

    return x, np.array(path)

# Comparison on Rosenbrock

def rosenbrock_grad(x):
    gx = -2*(1 - x[0]) - 400*x[0]*(x[1] - x[0]**2)
```

5.5 AdaGrad

Motivation: uniform step factor α is suboptimal when gradient components have very different magnitudes (e.g. sparse features in ML).

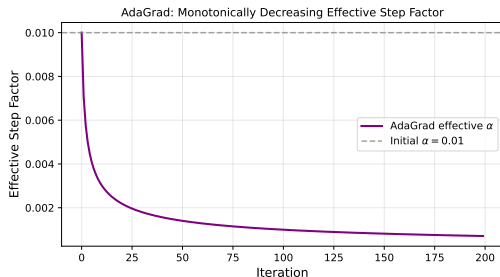
AdaGrad adapts a *componentwise* step factor based on the history of squared gradients.

AdaGrad Update (Eqs. 5.24–5.26)

$$s_i^{(k)} = \sum_{j=1}^k \left(g_i^{(j)}\right)^2 \quad (\text{accumulated squared gradient})$$

$$\alpha_{\text{adagrad},i}^{(k)} = \frac{\alpha}{\varepsilon + \sqrt{s_i^{(k)}}}$$

$$x_i^{(k+1)} = x_i^{(k)} - \alpha_{\text{adagrad},i}^{(k)} g_i^{(k)}$$



AdaGrad's effective step factor decays to zero. This is a primary weakness for long training runs.

Weakness

$s_i^{(k)}$ only grows $\Rightarrow$ effective step $\rightarrow 0$ before convergence. Fine for sparse problems; problematic for dense gradients.

5.5 AdaGrad — Python Implementation

```
import numpy as np

def adagrad(grad_f, x, alpha=0.01, eps=1e-8, n_iter=500):
    """
    AdaGrad: adapts per-dimension step factors.
    grad_f : gradient function
    x       : initial point
    alpha   : global step factor (default 0.01)
    eps     : small constant to avoid division by zero
    """
    s = np.zeros_like(x)      # accumulated squared gradients
    path = [x.copy()]

    for _ in range(n_iter):
        g = grad_f(x)
        s += g**2              # accumulate
        x = x - alpha / (np.sqrt(s) + eps) * g
        path.append(x.copy())

    return x, np.array(path)

# Running example

# f(x) = x1^2 + 10*x2^2 (ill-conditioned, ratio 10:1)
A = np.diag([2.0, 20.0])
grad_f = lambda x: A @ x
```


5.6 RMSProp

RMSProp (Tieleman & Hinton, 2012) fixes AdaGrad's monotone decay by using an **exponential moving average** of squared gradients.

RMSProp Update (Algorithm 5.6)

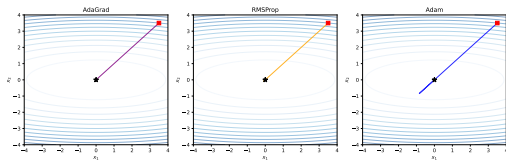
$$\mathbf{s}^{(k)} = \gamma \mathbf{s}^{(k-1)} + (1 - \gamma) \mathbf{g}^{(k)} \odot \mathbf{g}^{(k)}$$
$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - \frac{\alpha}{\sqrt{\mathbf{s}^{(k)}} + \varepsilon} \odot \mathbf{g}^{(k)}$$

- $\gamma \in [0, 1)$: decay rate (typical 0.9)
- α : global step factor (typical 0.001)
- $\varepsilon \approx 10^{-8}$: numerical stability
- $\odot$: elementwise product/division
- $\mathbf{s}^{(1)} = \mathbf{0}$ (zero initialisation)

Advantage over AdaGrad: $\mathbf{s}$ can grow or shrink, so the effective step factor does not inevitably decay to

Comparison

	AdaGrad	RMSProp
$\mathbf{s}$ update	$\mathbf{s} += \mathbf{g}^2$	$\mathbf{s} = \gamma \mathbf{s} + (1 - \gamma) \mathbf{g}^2$
$\mathbf{s}$ trend	Nondecreasing	Stabilises
Sparse data	Excellent	Good
Long runs	Fails	Robust



AdaGrad (left), RMSProp (centre), Adam (right) on $f(\mathbf{x}) = x_1^2 + 10x_2^2$.

5.6 RMSProp — Python Implementation

```
import numpy as np

def rmsprop(grad_f, x, alpha=0.001, gamma=0.9,
            eps=1e-8, n_iter=500):
    """
    RMSProp: exponential moving average of squared gradients.
    grad_f : gradient function
    x       : initial point
    alpha   : global learning rate
    gamma    : decay rate for squared gradient average
    eps     : small constant for numerical stability
    """
    s = np.zeros_like(x)      # squared gradient EMA
    path = [x.copy()]

    for _ in range(n_iter):
        g = grad_f(x)
        s = gamma * s + (1 - gamma) * g**2    # EMA update
        x = x - alpha / (np.sqrt(s) + eps) * g
        path.append(x.copy())

    return x, np.array(path)

# Example

grad_f = lambda x: np.array([2*x[0], 20*x[1]]) # f = x1^2+10*x2^2
```

5.7 Adadelta

Adadelta (Zeiler, 2012) eliminates the need for a global learning rate α entirely, using a ratio of two exponential moving averages.

Adadelta Update (Algorithm 5.7)

Track **two** EMAs:

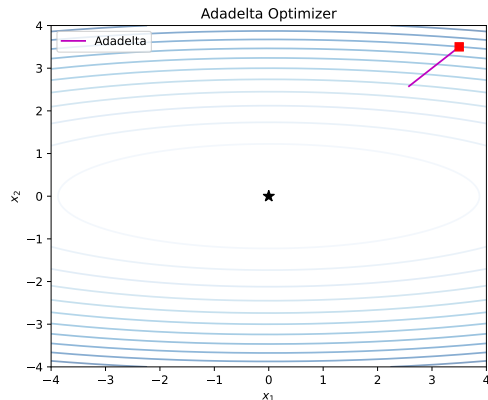
$$\mathbf{s}^{(k)} = \gamma_s \mathbf{s}^{(k-1)} + (1 - \gamma_s) \mathbf{g}^{(k)} \odot \mathbf{g}^{(k)}$$

$$\Delta \mathbf{x}^{(k)} = - \frac{\sqrt{\mathbf{u}^{(k-1)} + \varepsilon}}{\sqrt{\mathbf{s}^{(k)} + \varepsilon}} \odot \mathbf{g}^{(k)}$$

$$\mathbf{u}^{(k)} = \gamma_x \mathbf{u}^{(k-1)} + (1 - \gamma_x) \Delta \mathbf{x}^{(k)} \odot \Delta \mathbf{x}^{(k)}$$

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \Delta \mathbf{x}^{(k)}$$

- **s**: EMA of squared gradients
- **u**: EMA of squared updates
- ε added to numerator & denominator (prevents first step where



Key insight: ratio $\sqrt{\mathbf{u}}/\sqrt{\mathbf{s}}$ has units of $\Delta \mathbf{x}/g$, matching the Hessian approximation. This gives unit-invariant updates.

Typical hyperparameters

5.7 Adadelta — Python Implementation

```
import numpy as np

def adadelta(grad_f, x, gamma_s=0.95, gamma_x=0.95,
             eps=1e-6, n_iter=500):
    """
    Adadelta: no global learning rate required.
    grad_f   : gradient function
    x        : initial point
    gamma_s  : decay for squared gradient EMA
    gamma_x  : decay for squared update EMA
    eps      : small constant (added to BOTH numerator and denominator)
    """
    s = np.zeros_like(x)    # EMA of squared gradients
    u = np.zeros_like(x)    # EMA of squared updates
    path = [x.copy()]

    for _ in range(n_iter):
        g = grad_f(x)
        # Update squared-gradient EMA
        s = gamma_s * s + (1 - gamma_s) * g**2
        # Compute update using RMS ratio
        dx = -(np.sqrt(u + eps) / np.sqrt(s + eps)) * g
        # Update squared-update EMA
        u = gamma_x * u + (1 - gamma_x) * dx**2
        x = x + dx
        path.append(x.copy())
```

5.8 Adam

Adam (Kingma & Ba, 2014) = **A**daptive **M**oment estimation. Combines momentum with adaptive learning rates.

Adam Update (Algorithm 5.8)

Track biased **1st moment** (mean) and **2nd moment** (variance) of gradients:

$$\mathbf{v}^{(k)} = \gamma_v \mathbf{v}^{(k-1)} + (1 - \gamma_v) \mathbf{g}^{(k)}$$

$$\mathbf{s}^{(k)} = \gamma_s \mathbf{s}^{(k-1)} + (1 - \gamma_s) \mathbf{g}^{(k)} \odot \mathbf{g}^{(k)}$$

Bias correction (Eqs. 5.30–5.31):

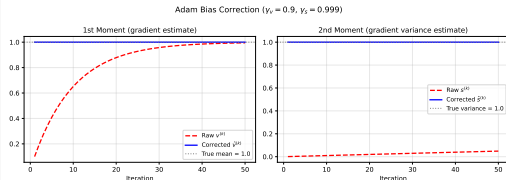
$$\hat{\mathbf{v}}^{(k)} = \frac{\mathbf{v}^{(k)}}{1 - \gamma_v^k}, \quad \hat{\mathbf{s}}^{(k)} = \frac{\mathbf{s}^{(k)}}{1 - \gamma_s^k}$$

Update:

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - \alpha \frac{\hat{\mathbf{v}}^{(k)}}{\sqrt{\hat{\mathbf{s}}^{(k)}} + \epsilon}$$

Default hyperparameters

$\alpha = 0.001$	learning rate
$\gamma_v = 0.9$	1st moment decay
$\gamma_s = 0.999$	2nd moment decay
$\epsilon = 10^{-8}$	stability constant
$\mathbf{v}^{(0)} = \mathbf{0}$	initial 1st moment
$\mathbf{s}^{(0)} = \mathbf{0}$	initial 2nd moment



Bias correction brings estimates to the true values during the critical early iterations.

5.8 Adam — Python Implementation

```
import numpy as np

def adam(grad_f, x, alpha=0.001, gamma_v=0.9, gamma_s=0.999,
         eps=1e-8, n_iter=1000):
    """
    Adam: Adaptive Moment Estimation (Kingma & Ba, 2014).
    grad_f : gradient function
    x       : initial point
    alpha   : global learning rate
    gamma_v : 1st moment decay (default 0.9)
    gamma_s : 2nd moment decay (default 0.999)
    eps     : numerical stability constant
    """
    v = np.zeros_like(x)    # 1st moment (mean)
    s = np.zeros_like(x)    # 2nd moment (uncentered variance)
    path = [x.copy()]

    for k in range(1, n_iter + 1):
        g = grad_f(x)
        v = gamma_v * v + (1 - gamma_v) * g           # biased 1st moment
        s = gamma_s * s + (1 - gamma_s) * g**2        # biased 2nd moment
        v_hat = v / (1 - gamma_v**k)                 # bias-corrected
        s_hat = s / (1 - gamma_s**k)                 # bias-corrected
        x = x - alpha * v_hat / (np.sqrt(s_hat) + eps)
        path.append(x.copy())

    return x, np.array(path)
```

5.8 Adam — Bias Correction Deep Dive

Why bias correction?

At initialisation $\mathbf{v}^{(0)} = \mathbf{0}$, $\mathbf{s}^{(0)} = \mathbf{0}$. After step 1:

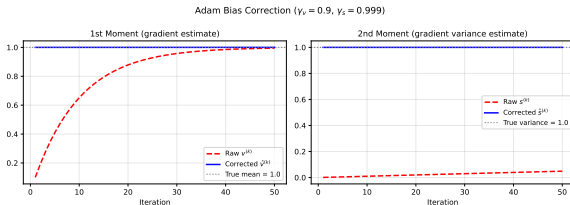
$$\mathbf{v}^{(1)} = (1 - \gamma_v) \mathbf{g}^{(1)}$$

Since $\gamma_v = 0.9 \Rightarrow \mathbf{v}^{(1)} = 0.1 \mathbf{g}^{(1)}$, severely underestimating the true gradient.

Bias correction restores the correct scale:

$$\hat{\mathbf{v}}^{(k)} = \frac{\mathbf{v}^{(k)}}{1 - \gamma_v^k} \xrightarrow{k \rightarrow \infty} \mathbf{v}^{(k)}$$

As k grows, $1 - \gamma_v^k \rightarrow 1$ so correction becomes negligible — only matters early in training.



Adam Algorithm Summary

Combines:

- Momentum ($\mathbf{v}$): accelerates in consistent directions
- AdaGrad/RMSProp ($\mathbf{s}$): scales by gradient history
- Bias correction: essential for first few iterations

Result: robust default for deep learning

5.9 Hypergradient Descent — Motivation

Problem: the step factor α in gradient descent must be manually tuned. Can we *learn* it automatically?

Idea: treat α itself as a variable and apply gradient descent *to it*.

Hypergradient (Eq. 5.35–5.36)

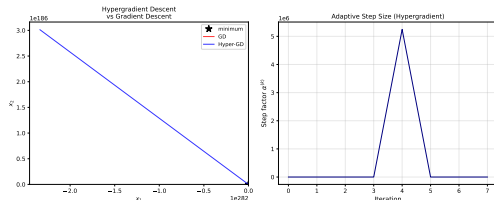
For gradient descent $\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - \alpha^{(k)} \mathbf{g}^{(k)}$:

$$\begin{aligned}\frac{\partial f(\mathbf{x}^{(k+1)})}{\partial \alpha^{(k)}} &= \left(\mathbf{g}^{(k+1)} \right)^\top \frac{\partial}{\partial \alpha^{(k)}} \left(\mathbf{x}^{(k)} - \alpha^{(k)} \mathbf{g}^{(k)} \right) \\ &= - \left(\mathbf{g}^{(k+1)} \right)^\top \mathbf{g}^{(k)}\end{aligned}$$

Only needs the *previous gradient* $\mathbf{g}^{(k)}$!

Step Size Update Rule (Eq. 5.37–5.38)

$$\alpha^{(k)} = \alpha^{(k-1)} + \mu \mathbf{g}^{(k)} \cdot \mathbf{g}^{(k-1)}$$



Left: hypergradient GD vs standard GD trajectories.
Right: the step size adapts over iterations.

Interpretation:

- $\mathbf{g}^{(k)} \cdot \mathbf{g}^{(k-1)} > 0$: α should increase (progress is consistent)
- $\mathbf{g}^{(k)} \cdot \mathbf{g}^{(k-1)} < 0$: α should decrease (oscillating, step too large)

5.9 Hypergradient Descent — Python Implementation

```
import numpy as np

def hypergradient_descent(grad_f, x, alpha0=0.001,
                          mu=1e-6, n_iter=500):
    """
    Hypergradient descent: gradient descent on the step factor.
    grad_f : gradient function
    x       : initial point
    alpha0  : initial step factor
    mu      : hypergradient step factor (meta-learning rate)
    """
    alpha = alpha0
    g_prev = np.zeros_like(x)
    path = [x.copy()]

    for _ in range(n_iter):
        g = grad_f(x)
        alpha = alpha + mu * np.dot(g, g_prev)  # update alpha
        alpha = max(alpha, 1e-10)              # keep positive
        g_prev = g.copy()
        x = x - alpha * g
        path.append(x.copy())

    return x, np.array(path)

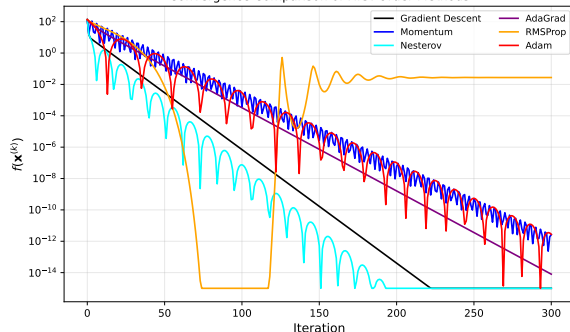
def hyper_nesterov(grad_f, x, alpha0=0.001, beta=0.9,
                    mu=1e-6, n_iter=500):
    """
    Hypergradient Nesterov: gradient descent on the step factor with Nesterov momentum.
    grad_f : gradient function
    x       : initial point
    alpha0  : initial step factor
    beta    : momentum factor
    mu      : hypergradient step factor (meta-learning rate)
    """
    alpha = alpha0
    g_prev = np.zeros_like(x)
    path = [x.copy()]

    for _ in range(n_iter):
        g = grad_f(x)
        alpha = alpha + mu * np.dot(g, g_prev)  # update alpha
        alpha = max(alpha, 1e-10)              # keep positive
        g_prev = g.copy()
        x = x - alpha * g
        path.append(x.copy())

    return x, np.array(path)
```

Convergence Comparison of All Methods

Convergence Comparison of First-Order Methods



$f(\mathbf{x}) = x_1^2 + 10x_2^2$, starting from $\mathbf{x}^{(0)} = [3.5, 3.5]$.

Method	Convergence	Notes
GD	$O(1/k)$	Baseline
Conjugate GD	$O(1/k^2)^*$	n steps exact
Momentum	$O(1/k)$	faster in practice
Nesterov	$O(1/k^2)$	optimal for convex f
AdaGrad	$O(1/\sqrt{k})$	sparse gradients
RMSProp	adaptive	non-convex
Adadelta	no α	unit-invariant
Adam	adaptive	default for DL
Hypergradient	adaptive	learns α

Practical takeaways:

- Adam is the de-facto standard for deep learning
- Nesterov is best for smooth convex problems
- CG converges in n steps for quadratics
- Hypergradient is useful when tuning α is hard

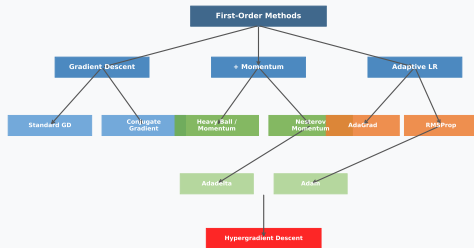
*For the conjugate gradient method on quadratics; behaviour on general f varies

5.10 Summary

Key Points

- **Gradient descent** follows the direction of steepest descent $-\nabla f / \|\nabla f\|$.
- **Conjugate gradient** automatically adjusts to local valleys; converges in n steps for quadratics.
- **Momentum** builds up velocity in consistent directions, damping oscillations.
- **Nesterov momentum** evaluates the gradient at the lookahead position, achieving $O(1/k^2)$ convergence.
- **AdaGrad** scales steps by accumulated gradient history — good for sparse gradients.
- **RMSProp** uses an EMA of squared gradients, preventing AdaGrad's step decay.
- **Adadelta** eliminates the global learning rate using a ratio of two EMAs

Taxonomy of First-Order Methods (Chapter 5)



Taxonomy of first-order methods.

5.11 Selected Exercises with Solutions I

Exercise 5.1

For $f(\mathbf{x}) = x_1 x_2^2$, at $\mathbf{x}^{(k)} = [1, 2]$, compute the normalised gradient descent direction.

Solution: $\nabla f = [x_2^2, 2x_1 x_2] = [4, 4]$. Unnormalized direction: $[-4, -4]$. Normalised: $\mathbf{d}^{(k)} = [-1/\sqrt{2}, -1/\sqrt{2}]$.

Exercise 5.3

Apply gradient descent with $\alpha = 1$ to $f(x) = x^4$, starting at $x^{(1)} = 1$.

Solution: $f'(x) = 4x^3$.

$$f'(1) = 4$$

$$\rightarrow x^{(2)} = 1 - 4 = -3$$

$$f'(-3) = 4(-27) = -108$$

$$\rightarrow x^{(3)} = -3 + 108 = 105$$

Step size $\alpha = 1$ is too large; iterates diverge. A smaller α (e.g. 0.01) is needed.

5.11 Selected Exercises with Solutions II

Exercise 5.8

CG on $f(x, y) = x^2 + xy + y^2 + 5$, starting at $(1, 1)$.

Solution: $\nabla f(x, y) = [2x + y, 2y + x]$. At $(1, 1)$: $\nabla f = [3, 3]$, so $\mathbf{d}^{(1)} = [-3, -3]$.

The Hessian $\mathbf{H} = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$ is positive definite. CG converges in ≤ 2 steps. The optimum is $(0, 0)$.

Exercise 5.6

How is Nesterov momentum an improvement over standard momentum?

Solution: Nesterov momentum evaluates the gradient at the *anticipated next position* $\mathbf{x}^{(k)} + \beta \mathbf{v}^{(k)}$, rather than at $\mathbf{x}^{(k)}$. This provides a more accurate gradient estimate for the update, leading to better convergence for convex f .

5.11 Selected Exercises with Solutions III

Exercise 5.10 (Heavy Ball)

Show that the heavy ball iterates are identical to those of momentum.

Solution: Define $\mathbf{v}^{(k)} = \mathbf{x}^{(k)} - \mathbf{x}^{(k-1)}$. Heavy ball: $\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - \alpha \mathbf{g}^{(k)} + \beta \mathbf{v}^{(k)}$. Then $\mathbf{v}^{(k+1)} = \mathbf{x}^{(k+1)} - \mathbf{x}^{(k)} = -\alpha \mathbf{g}^{(k)} + \beta \mathbf{v}^{(k)}$, which is exactly the momentum update rule.

Worked Example: Adam on a 2D Optimization Problem

Problem: minimise $f(\mathbf{x}) = x_1^2 + 10x_2^2$ from $\mathbf{x}^{(0)} = [3.5, 3.5]$ using Adam.

First few iterations (manually):

$$\mathbf{g}^{(1)} = [7.0, 70.0]$$

After bias correction at $k = 1$ ($\gamma_v = 0.9$, $\gamma_s = 0.999$):

$$\hat{v}_1 = \frac{0.1 \times 7}{0.1} = 7.0$$

$$\hat{s}_1 = \frac{0.001 \times 49}{0.001} = 49.0$$

$$x_1^{(2)} = 3.5 - 0.001 \times \frac{7.0}{\sqrt{49.0 + 10^{-8}}} = 3.5 - 0.001 \approx 3$$

Each step moves by exactly $\pm\alpha$ per dimension when $\hat{v}$ and $\sqrt{\hat{s}}$ match.

```
1 import numpy as np
2
3 def adam_trace(alpha=0.001, gamma_v=0.9,
4               gamma_s=0.999, eps=1e-8):
5     x = np.array([3.5, 3.5])
6     v = np.zeros(2); s = np.zeros(2)
7     A = np.diag([2.0, 20.0])
8
9     for k in range(1, 6):
10         g = A @ x
11         v = gamma_v*v + (1-gamma_v)*g
12         s = gamma_s*s + (1-gamma_s)*g**2
13         v_h = v/(1 - gamma_v**k)
14         s_h = s/(1 - gamma_s**k)
15         step = alpha*v_h/(np.sqrt(s_h)+eps)
16         x = x - step
17         print(f"k={k}: x={x.round(4)}, "
18               f"|g|={np.linalg.norm(g):.3f}")
19
20 adam_trace()
21 # k=1: x=[3.499 3.499], |g|=70.711
22 # k=2: x=[3.498 3.498], |g|=70.686
23 # ...converges to [0, 0]
```

Listing 10: Adam Step-by-Step Trace

Further Reading and Resources

Original Papers:

- **Conjugate Gradient:** Fletcher & Reeves, 1964
- **Momentum (Heavy Ball):** Polyak, 1964
- **Nesterov:** Nesterov, 1983
- **AdaGrad:** Duchi et al., 2011
- **RMSProp:** Tieleman & Hinton, 2012
- **Adadelta:** Zeiler, 2012
- **Adam:** Kingma & Ba, 2014
- **Hypergradient:** Baydin et al., 2018

Related Textbook Sections:

- Ch. 2: Derivatives and Gradients
- Ch. 4: Descent Direction Methods
- Appendix B.6: Rosenbrock function
- Appendix C.5: Subgradients

Python Ecosystem

Method	Python Package
All methods	<code>torch.optim</code> (PyTorch)
Adam, etc.	<code>tensorflow.keras.optimizers</code>
L-BFGS (CG-like)	<code>scipy.optimize.minimize</code>
Custom	<code>numpy</code> (as shown)

scipy Example

```
1 f = lambda x: x[0]**2 + 10*x[1]**2 gradf = lambda x:  
  np.array([2*x[0], 20*x[1]])  
2 Conjugate-gradient method in scipy res = minimize(f, [3.5,  
  3.5], jac=gradf, method='CG') print(res.x) [0., 0.]  
3 ~
```